



University of Engineering & Technology Peshawar

Computer Programming (C++) Lab

Assginment 02

STUDENT NAME	Muhammad Yaseen
REGISTRATION NO:	BFNWELE0670
COUSRE:	EE-170L COMPUTER PROGRAMMING LAB
DATE:	28/04/26
Assginment	02

SUBMITTED TO ENGR RIZWAN ULLAH

Q1 – Array Statistics (Max, Min, Average)

Description: Takes 10 integer inputs in an array and finds the maximum value, minimum value, and average.

Source Code (C++):

```
#include <iostream>
using namespace std;

int main() {
    int arr[10];
    cout << "Enter 10 integers: " << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Element [" << i + 1 << "]: ";
        cin >> arr[i];
    }

    int maxVal = arr[0], minVal = arr[0];
    double sum = 0;

    for (int i = 0; i < 10; i++) {
        if (arr[i] > maxVal) maxVal = arr[i];
        if (arr[i] < minVal) minVal = arr[i];
        sum += arr[i];
    }

    cout << "\n--- Results ---" << endl;
    cout << "Maximum Value : " << maxVal << endl;
    cout << "Minimum Value : " << minVal << endl;
    cout << "Average      : " << sum / 10 << endl;

    return 0;
}
```

Q2 – Reverse a 1D Array Using a Function

Description: Uses a function to reverse a 1D array and displays the array before and after reversal.

Source Code (C++):

```
#include <iostream>
using namespace std;

void reverseArray(int arr[], int size) {
    int start = 0, end = size - 1;
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}

void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i];
        if (i < size - 1) cout << " ";
    }
    cout << endl;
}

int main() {
    const int SIZE = 6;
    int arr[SIZE];

    cout << "Enter " << SIZE << " integers:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Element [" << i + 1 << "]: ";
        cin >> arr[i];
    }

    cout << "\nArray BEFORE reversal: ";
    displayArray(arr, SIZE);

    reverseArray(arr, SIZE);

    cout << "Array AFTER reversal: ";
    displayArray(arr, SIZE);

    return 0;
}
```

Q3 – 3×3 Matrix Addition

Description: Inputs two 3×3 matrices, performs matrix addition, and displays the result matrix.

Source Code (C++):

```
#include <iostream>
using namespace std;

void inputMatrix(int mat[3][3], const string& name) {
    cout << "Enter elements of Matrix " << name << " (3x3):" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++) {
            cout << " [" << i << "][" << j << "]: ";
            cin >> mat[i][j];
        }
}

void displayMatrix(int mat[3][3]) {
    for (int i = 0; i < 3; i++) {
        cout << " | ";
        for (int j = 0; j < 3; j++)
            cout << mat[i][j] << "\t";
        cout << "|" << endl;
    }
}

void addMatrices(int a[3][3], int b[3][3], int result[3][3]) {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            result[i][j] = a[i][j] + b[i][j];
}

int main() {
    int A[3][3], B[3][3], C[3][3];

    inputMatrix(A, "A");
    inputMatrix(B, "B");
    addMatrices(A, B, C);

    cout << "\nMatrix A:" << endl;
    displayMatrix(A);
    cout << "Matrix B:" << endl;
    displayMatrix(B);
    cout << "Result (A + B):" << endl;
    displayMatrix(C);

    return 0;
}
```

Q4 – Alphabetical Sorting of Student Names

Description: Stores names of 5 students in a string array, sorts them alphabetically using bubble sort, and displays the sorted list.

Source Code (C++):

```
#include <iostream>
#include <string>
using namespace std;

void bubbleSort(string names[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (names[j] > names[j + 1]) {
                string temp = names[j];
                names[j] = names[j + 1];
                names[j + 1] = temp;
            }
        }
    }
}

int main() {
    const int SIZE = 5;
    string names[SIZE];

    cout << "Enter names of 5 students:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Student " << i + 1 << ": ";
        cin >> names[i];
    }

    bubbleSort(names, SIZE);

    cout << "\nSorted names (Alphabetical Order):" << endl;
    for (int i = 0; i < SIZE; i++)
        cout << i + 1 << ". " << names[i] << endl;

    return 0;
}
```

Q5 – Power System Load Analysis (1D Array + Functions)

Description: Stores hourly load demand (MW) for 24 hours. Functions calculate total load, peak load, and average load.

Source Code (C++):

```
#include <iostream>
using namespace std;

double calculateTotalLoad(double load[], int size) {
    double total = 0;
    for (int i = 0; i < size; i++)
        total += load[i];
    return total;
}

double calculatePeakLoad(double load[], int size, int& peakHour) {
    double peak = load[0];
    peakHour = 0;
    for (int i = 1; i < size; i++) {
        if (load[i] > peak) {
            peak = load[i];
            peakHour = i;
        }
    }
    return peak;
}

int main() {
    const int HOURS = 24;
    double load[HOURS];

    cout << "Enter hourly load demand (MW) for 24 hours:" << endl;
    for (int i = 0; i < HOURS; i++) {
        cout << "  Hour " << (i < 9 ? " " : "") << i + 1 << " : ";
        cin >> load[i];
    }

    int peakHour;
    double total = calculateTotalLoad(load, HOURS);
    double peak = calculatePeakLoad(load, HOURS, peakHour);

    cout << "\n==== Power System Load Analysis =====" << endl;
    cout << "Total Load Demand : " << total << " MW" << endl;
    cout << "Peak Load      : " << peak << " MW (at Hour " <<
peakHour + 1 << ")" << endl;
    cout << "Average Load      : " << total / HOURS << " MW" << endl;

    return 0;
}
```

Q6 – Student Record System (Names, Marks, Search)

Description: Stores student names and marks. Functions find the topper, display pass/fail status (pass mark = 50), and search a student by name.

Source Code (C++):

```
#include <iostream>
#include <string>
using namespace std;

const int SIZE = 5;
const int PASS_MARKS = 50;

void findTopper(string names[], int marks[]) {
    int maxIdx = 0;
    for (int i = 1; i < SIZE; i++)
        if (marks[i] > marks[maxIdx]) maxIdx = i;
    cout << "Topper: " << names[maxIdx] << " with " << marks[maxIdx] << "
marks" << endl;
}

void displayPassFail(string names[], int marks[]) {
    cout << "\n--- Pass/Fail Status ---" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << names[i] << " (" << marks[i] << "): ";
        cout << (marks[i] >= PASS_MARKS ? "PASS" : "FAIL") << endl;
    }
}

void searchStudent(string names[], int marks[]) {
    string target;
    cout << "\nEnter student name to search: ";
    cin >> target;

    bool found = false;
    for (int i = 0; i < SIZE; i++) {
        if (names[i] == target) {
            cout << "Found! " << names[i] << " -> Marks: " << marks[i] <<
endl;
            found = true;
            break;
        }
    }
    if (!found) cout << "Student \" " << target << "\" not found." <<
endl;
}

int main() {
    string names[SIZE];
    int marks[SIZE];

    cout << "Enter student names and marks:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Student " << i + 1 << " Name : ";
        cin >> names[i];
        cout << "Student " << i + 1 << " Marks: ";
        cin >> marks[i];
    }

    cout << "\n--- Topper ---" << endl;
    findTopper(names, marks);
    displayPassFail(names, marks);
    searchStudent(names, marks);

    return 0;
}
```

Q7 – Renewable Energy Monitoring System (Solar Plant)

Description: Stores solar energy generation (kWh) for 24 hours. Functions calculate total energy, the hour with maximum generation, and average hourly output.

Source Code (C++):

```
#include <iostream>
using namespace std;

const int HOURS = 24;

double calculateTotal(double energy[]) {
    double total = 0;
    for (int i = 0; i < HOURS; i++)
        total += energy[i];
    return total;
}

int findPeakHour(double energy[]) {
    int peak = 0;
    for (int i = 1; i < HOURS; i++)
        if (energy[i] > energy[peak]) peak = i;
    return peak;
}

double calculateAverage(double energy[]) {
    return calculateTotal(energy) / HOURS;
}

void displayResults(double energy[]) {
    double total = calculateTotal(energy);
    int peak = findPeakHour(energy);
    double avg = calculateAverage(energy);

    cout << "\n==== Solar Energy Monitoring Report =====> endl;
    cout << "Total Energy Generated : " << total << " kWh" << endl;
    cout << "Peak Generation Hour : Hour " << peak + 1 << " (" <<
energy[peak] << " kWh)" << endl;
    cout << "Average Hourly Output : " << avg << " kWh" << endl;

    cout << "\n--- Hourly Breakdown ---" << endl;
    for (int i = 0; i < HOURS; i++)
        cout << "Hour " << (i < 9 ? " " : "") << i + 1 << " : " <<
energy[i] << " kWh" << endl;
}

int main() {
    double energy[HOURS];

    cout << "Enter solar energy generated (kWh) for each of the 24 hours:"
<< endl;
    for (int i = 0; i < HOURS; i++) {
        cout << " Hour " << (i < 9 ? " " : "") << i + 1 << " : ";
        cin >> energy[i];
    }

    displayResults(energy);
    return 0;
}
```


Q8 – Battery Health Monitoring System

Description: Stores 20 voltage readings. Safe limits: 3.0V – 4.2V. Functions count unsafe readings, find max/min voltage, and display warnings if unsafe readings exist.

Source Code (C++):

```
#include <iostream>
using namespace std;

const int    READINGS    = 20;
const double MIN_VOLT    = 3.0;
const double MAX_VOLT    = 4.2;

void countUnsafeReadings(double voltage[], int& below, int& above) {
    below = 0; above = 0;
    for (int i = 0; i < READINGS; i++) {
        if (voltage[i] < MIN_VOLT) below++;
        else if (voltage[i] > MAX_VOLT) above++;
    }
}

void findMaxMin(double voltage[], double& maxV, double& minV) {
    maxV = voltage[0];
    minV = voltage[0];
    for (int i = 1; i < READINGS; i++) {
        if (voltage[i] > maxV) maxV = voltage[i];
        if (voltage[i] < minV) minV = voltage[i];
    }
}

int main() {
    double voltage[READINGS];

    cout << "Enter " << READINGS << " voltage readings (V):" << endl;
    for (int i = 0; i < READINGS; i++) {
        cout << "  Reading " << (i < 9 ? " " : "") << i + 1 << " : ";
        cin >> voltage[i];
    }

    int below, above;
    double maxV, minV;
    countUnsafeReadings(voltage, below, above);
    findMaxMin(voltage, maxV, minV);

    cout << "\n==== Battery Health Report =====" << endl;
    cout << "Safe Voltage Range      : " << MIN_VOLT << "V - " << MAX_VOLT
    << "V" << endl;
    cout << "Maximum Voltage          : " << maxV << " V" << endl;
    cout << "Minimum Voltage          : " << minV << " V" << endl;
    cout << "Readings Below Limit     : " << below << endl;
    cout << "Readings Above Limit     : " << above << endl;

    if (below > 0 || above > 0) {
        cout << "\n*** WARNING: Unsafe voltage readings detected! ***" <<
        endl;
        if (below > 0) cout << "  -> " << below << " reading(s) are BELOW
        the safe threshold (" << MIN_VOLT << "V)." << endl;
        if (above > 0) cout << "  -> " << above << " reading(s) are ABOVE
        the safe threshold (" << MAX_VOLT << "V)." << endl;
        cout << "  -> Immediate inspection is recommended." << endl;
    } else {
        cout << "\nAll readings are within safe limits." << endl;
    }

    return 0;
}
```

Q9 – Smart Grid Load Distribution (2D Array + Functions)

Description: Stores power consumption (MW) for 4 regions over 7 days in a 2D array. Functions calculate total per region, identify the peak demand day, and compute overall average.

Source Code (C++):

```
#include <iostream>
using namespace std;

const int REGIONS = 4;
const int DAYS = 7;

void calcRegionTotals(double grid[REGIONS][DAYS], double totals[]) {
    for (int r = 0; r < REGIONS; r++) {
        totals[r] = 0;
        for (int d = 0; d < DAYS; d++)
            totals[r] += grid[r][d];
    }
}

int findPeakDay(double grid[REGIONS][DAYS]) {
    int peakDay = 0;
    double maxDemand = 0;
    for (int d = 0; d < DAYS; d++) {
        double dayTotal = 0;
        for (int r = 0; r < REGIONS; r++)
            dayTotal += grid[r][d];
        if (dayTotal > maxDemand) {
            maxDemand = dayTotal;
            peakDay = d;
        }
    }
    return peakDay;
}

double calcOverallAverage(double grid[REGIONS][DAYS]) {
    double total = 0;
    for (int r = 0; r < REGIONS; r++)
        for (int d = 0; d < DAYS; d++)
            total += grid[r][d];
    return total / (REGIONS * DAYS);
}

int main() {
    double grid[REGIONS][DAYS];
    string dayNames[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

    cout << "Enter power consumption (MW) for 4 regions over 7 days:" <<
endl;
    for (int r = 0; r < REGIONS; r++) {
        cout << "\nRegion " << r + 1 << ":" << endl;
        for (int d = 0; d < DAYS; d++) {
            cout << "    " << dayNames[d] << ": ";
            cin >> grid[r][d];
        }
    }

    double regionTotals[REGIONS];
    calcRegionTotals(grid, regionTotals);
    int peakDay = findPeakDay(grid);
    double avg = calcOverallAverage(grid);

    cout << "\n==== Smart Grid Load Distribution Report =====" << endl;
    cout << "\n--- Total Consumption per Region ---" << endl;
    for (int r = 0; r < REGIONS; r++)
        cout << "Region " << r + 1 << " : " << regionTotals[r] << " MW" <<
```

```
endl;

    cout << "\nDay with Highest Total Demand : " << dayNames[peakDay] << "
(Day " << peakDay + 1 << ")" << endl;
    cout << "Overall Average Consumption    : " << avg << " MW" << endl;

    return 0;
}
```

Q10 – Classroom Attendance System (2D Array + Functions)

Description: Stores attendance (1=Present, 0=Absent) for 5 students over 7 days in a 2D array. Functions calculate each student's total attendance, find the top attendee, and compute overall class attendance percentage.

Source Code (C++):

```
#include <iostream>
#include <string>
using namespace std;

const int STUDENTS = 5;
const int DAYS = 7;

void calcAttendance(int att[STUDENTS][DAYS], int totals[]) {
    for (int s = 0; s < STUDENTS; s++) {
        totals[s] = 0;
        for (int d = 0; d < DAYS; d++)
            totals[s] += att[s][d];
    }
}

int findTopAttendee(int totals[]) {
    int topIdx = 0;
    for (int i = 1; i < STUDENTS; i++)
        if (totals[i] > totals[topIdx]) topIdx = i;
    return topIdx;
}

double calcClassPercentage(int att[STUDENTS][DAYS]) {
    int present = 0;
    for (int s = 0; s < STUDENTS; s++)
        for (int d = 0; d < DAYS; d++)
            present += att[s][d];
    return (double)present / (STUDENTS * DAYS) * 100.0;
}

int main() {
    int att[STUDENTS][DAYS];
    string names[STUDENTS];
    string dayNames[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

    cout << "Enter student names:" << endl;
    for (int s = 0; s < STUDENTS; s++) {
        cout << "Student " << s + 1 << ": ";
        cin >> names[s];
    }

    cout << "\nEnter attendance (1=Present, 0=Absent):" << endl;
    for (int s = 0; s < STUDENTS; s++) {
        cout << "\n" << names[s] << ":" << endl;
        for (int d = 0; d < DAYS; d++) {
            cout << " " << dayNames[d] << " : ";
            cin >> att[s][d];
        }
    }
}
```

```
        while (att[s][d] != 0 && att[s][d] != 1) {
            cout << "    Invalid! Enter 0 or 1: ";
            cin >> att[s][d];
        }
    }

    int totals[STUDENTS];
    calcAttendance(att, totals);
    int topIdx = findTopAttendee(totals);
    double classPercent = calcClassPercentage(att);

    cout << "\n==== Classroom Attendance Report =====> endl;
    cout << "\n--- Individual Attendance (out of " << DAYS << " days) ---"
<< endl;
    for (int s = 0; s < STUDENTS; s++) {
        double pct = (double)totals[s] / DAYS * 100.0;
        cout << names[s] << " : " << totals[s] << " days present (" << pct
<< "%)" << endl;
    }

    cout << "\nHighest Attendance : " << names[topIdx] << " (" <<
totals[topIdx] << "/" << DAYS << " days)" << endl;
    cout << "Overall Class Attendance : " << classPercent << "%" << endl;

    return 0;
}
```

BFNWLE0670